CSA17 – Artificial Intelligence Assignment 2 – CO2

Informed search · Local search and optimisation · Online search · Constraint satisfaction · Adversarial search

Item	Details
Name	P . Lakshmi Gayathri Navaneetha
Register number	192321169
Programme / Department	Csa17-AI/ IT
Submission date	07-11-2026
Nature of assignment	Individual – 50 marks

Method used throughout: **Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate.** Operation counts (nodes expanded, constraint checks, evaluation calls, tours counted) are the primary evidence; execution time is supporting evidence only. All code is written in Python 3 (standard library, NumPy and Matplotlib for figures); every number in this report is produced by the accompanying scripts and stored in the raw-data files.

Table 1. Overview of the report.

Task	Topic	Marks	Main evidence
1	Uniform-cost, greedy and A* on four maps; heuristic proof	6	Metric tables, expansion maps, density sweep, 95% open-road profile
2	Online DFS and LRTA* rover	4	Distance, size, dead-end, unreachable and repeated-mission experiments
3	Alpha-beta under engineered tree orderings	6	Predicted vs observed leaves, nodes and cut-offs; depth sweep
4	Depth-limited minimax and scalability	6	Node-count derivation matched exactly; strength tests; 1 s limit
5	Constraint propagation derived from a principle	7	Pseudocode, domain-shrinking figures, BT vs MAC growth, soundness proof
6	Exhaustive vs local search (delivery tour)	8	n! validation, 2-opt gap and time, deployment boundary
7	Diagnose, redesign and validate (minimax)	13	Redundancy measurements, five-variant ablation, objective scorecard

Task 1 – Greedy Best-First and A* Under Different Map Conditions (6 marks)

1.1 Question and approach

The shuttle planner may use uniform-cost search (UCS), greedy best-first search (GBFS) or A*. The question is whether one of them should **always** be preferred, or whether the right choice depends on the map. I treat it as an experiment: predict first, design a heuristic and maps that can expose differences, implement all three algorithms with identical instrumentation, and compare observed behaviour with the predictions.

Model used: 4-connected grid, every move costs 1, # cells are blocked. The three algorithms differ only in the priority used to pick the next node from the frontier: **UCS** uses g(n), **GBFS** uses h(n), **A∗** uses g(n)+h(n).

1.2 Predictions made before running any code

Table 2. Predictions recorded before implementation.

Map	UCS	Greedy best-first	A*	Algorithmic reason
Open 40×40, corner to corner	Expands ≈ the whole grid (every cell has g ≤ 78)	≈ path length (about 79); optimal	≈ 79 when ties are broken towards smaller h; optimal	h decreases on every step of a shortest path; UCS has no sense of direction and grows a diamond over the whole grid.
Maze 41×41 (perfect maze)	Optimal; visits most corridors	Optimal (unique path); somewhat fewer nodes	Optimal; ≈ UCS because h is a poor guide	Only one route exists, so every algorithm must try dead ends; Manhattan distance is far below the true cost (≈ 500), so f barely discriminates.
Dead-end trap (U-shaped cup facing away from goal)	Most nodes; optimal	Fills the cup, **sub-optimal** path	Optimal; more nodes than greedy	h keeps pulling greedy towards the back wall of the cup; g stops A* being fooled but it still pays for exploring.
Cluttered 40×40 (25% random obstacles)	Most nodes; optimal	Fewest nodes, often sub-optimal	In between; optimal	Local obstacles make h misleading for greedy; A* is protected by g.

1.3 Heuristic design and proof of admissibility

Heuristic: $h(n) = |r(n) - r(goal)| + |c(n) - c(goal)|$ (Manhattan distance).

Relaxed problem: delete the rule "a move may only enter a free cell". In the relaxed problem the shuttle can drive straight over obstacles, and a move still changes exactly one coordinate by 1. The cheapest relaxed solution therefore needs exactly $|\Delta r|+|\Delta c|$ moves, i.e. the optimal cost of the relaxed problem is the Manhattan distance.

Admissibility: every real path is also a legal path in the relaxed problem (it obeys fewer rules), so the true optimal cost $h^*(n)$ is never smaller than the relaxed optimum: $h(n) \le h^*(n)$. The heuristic therefore never over-estimates. **Consistency:** for neighbouring cells n, n′ the Manhattan distances differ by at most 1 = c(n,n′), so $h(n) \le c(n,n') + h(n')$ and f = g+h never decreases along a path. A* returns an optimal path and never has to re-open a closed node (my code still allows re-opening for safety).

1.4 Map design

Table 3. Maps used in the experiments.

Map	Size / obstacles	Purpose
Open (contrasting map 1)	40×40, none; start (0,0), goal (39,39)	Best case for a directional heuristic; shows how much work UCS wastes.

Map	Size / obstacles	Purpose
Maze (contrasting map 2)	41×41 perfect maze from recursive back-tracker; start (1,1), goal (39,39)	Heuristic is nearly useless: corridors lead away from the goal; one optimal path of cost 508.
Dead-end trap (edge case)	40×40 with a U-shaped cup (walls on rows 8 and 32, columns 8–20, back wall column 20); start inside the cup (20,14), goal (20,33) behind the wall	Classic local-minimum trap: straight-line direction points into a dead end; optimal route must first move **away** from the goal.
Cluttered (extra)	40×40, 25% random obstacles (solvable)	Intermediate condition between open and maze.

1.5 Implementation and measures

One search routine serves all three algorithms; only the heap key differs. Ties are broken towards the node closer to the goal (smaller h) – without this, A* on an open map has f = 78 for every cell in the rectangle and expands all of them. Recorded per run: **nodes expanded** (popped and processed), **nodes generated** (pushed), **maximum frontier size**, **path cost**, **optimality** (compared with the UCS cost, which is optimal by construction) and **time** (median of 5 runs, additional evidence only).

Core of search() in task1.py

```
key = {"ucs":    lambda g,s: (g, h(s)),
       "greedy": lambda g,s: (h(s), -g),
       "astar":  lambda g,s: (g + h(s), h(s))}[algo]
heap = [(key(0,start), 0, start)];  best_g = {start: 0}
while heap:
    _, _, g, s = heappop(heap)
    if s in closed or g > best_g[s]: continue      # stale entry
    closed.add(s); expanded += 1
    if s == goal: return path, g
    for n in neighbours(s):                       # 4-connected, not a wall
        ng = g + 1
        if n not in best_g or ng < best_g[n]:      # greedy: only if n unseen
            best_g[n] = ng; parent[n] = s; push(n); generated += 1
```

1.6 Results on the designed maps

Table 4. Measured results (one run per map; deterministic).

Map	Algorithm	Expanded	Generated	Max frontier	Path cost	Optimal?	Time (ms)
Open	Uniform-cost	1,600	1,600	40	78	Yes	3.42
	Greedy best-first	79	155	77	78	Yes	0.20
	A*	79	155	77	78	Yes	0.22
Maze	Uniform-cost	763	765	5	508	Yes	1.36
	Greedy best-first	628	641	14	508	Yes	1.14
	A*	756	758	4	508	Yes	1.46
Dead-end trap	Uniform-cost	1,502	1,515	42	59	Yes	3.51
	Greedy best-first	460	526	67	69	**No** (opt. 59)	1.02
	A*	460	526	67	59	Yes	1.25
Cluttered (25%)	Uniform-cost	1,164	1,166	38	78	Yes	2.55
	Greedy best-first	133	219	87	114	**No** (opt. 78)	0.36
	A*	323	498	99	78	Yes	0.99

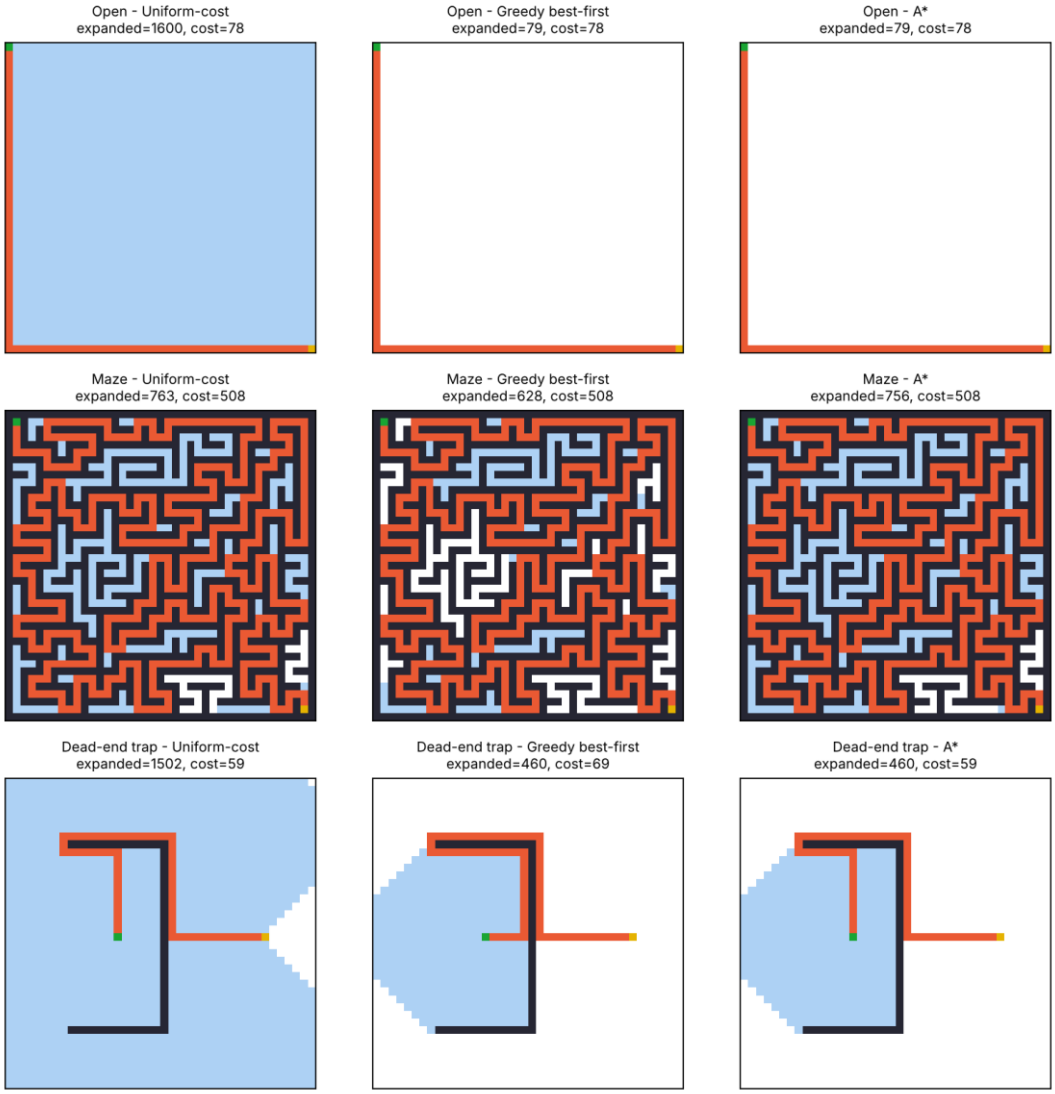


Figure 1. Cells expanded (light blue) and path found (red) for each algorithm on the open, maze and trap maps. Green = start, yellow = goal.

1.7 Prediction versus observation

Table 5. Scoreboard of predictions.

Map	Predicted	Observed	Verdict
Open	UCS ≈ whole grid; GBFS ≈ A* ≈ 79	UCS 1,600 (all cells); GBFS 79; A* 79; all optimal	**Confirmed** – UCS expands 20× more nodes for the same path.
Maze	All optimal; A* ≈ UCS; GBFS fewer	UCS 763, A* 756, GBFS 628; all cost 508	**Confirmed** – A* saves only 1% over UCS in the maze; greedy saves 18% but only because the path is unique.
Trap	GBFS sub-optimal; A* optimal with **more** nodes than GBFS; UCS most	GBFS cost 69 vs optimal 59 (17% longer); A* optimal; UCS 1,502; A* and GBFS both expand **460**	**Mostly confirmed.** The A*-more-than-greedy part was **wrong**: both expanded exactly the same 460 cells (I checked the two sets are identical). A* must expand every node with f < C* = 59; in this map that is precisely the region greedy also floods before escaping the cup.
Cluttered	GBFS fewest but often sub-optimal; A* between; UCS most	GBFS 133 (cost 114 vs 78, 46% longer); A* 323; UCS 1,164	**Confirmed.**

The single-map results could be a coincidence of one layout, so I also swept the obstacle density on 40×40 grids (60 random start/goal pairs per density, each at least 40 steps apart, always solvable) and the grid size on open maps.

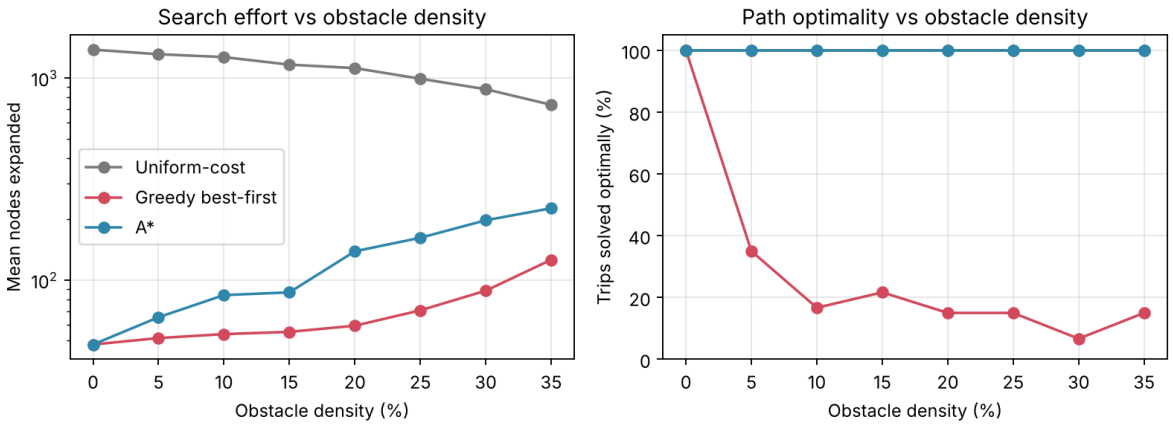


Figure 2. Effect of obstacle density on effort (log scale) and on the percentage of trips for which each algorithm returns an optimal path.

Table 6. Density sweep (means over 60 trips per row; A* and UCS were optimal in 100% of trips).

Obstacle density	UCS expanded	Greedy expanded	A* expanded	Greedy optimal trips	Greedy cost ÷ optimal
0.0%	1,377	48	48	100.0%	1.00
5.0%	1,308	52	65	35.0%	1.04
10.0%	1,265	54	84	16.7%	1.08
15.0%	1,161	55	87	21.7%	1.11
20.0%	1,116	59	138	15.0%	1.12
25.0%	989	71	161	15.0%	1.16
30.0%	878	88	197	6.7%	1.20
35.0%	735	126	226	15.0%	1.18

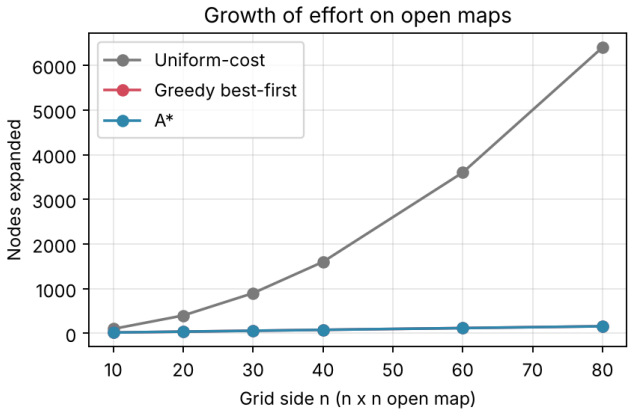


Figure 3. Open maps: UCS grows with the area (n^2 = 6,400 expansions at n=80) while GBFS and A* grow with the path length (159).

Reading the evidence. (i) UCS effort follows the free area, regardless of obstacles. (ii) A* effort follows the path length when the heuristic is informative and rises smoothly with clutter (48 → 226 expansions between 0% and 35% obstacles), never beyond UCS. (iii) Greedy is the cheapest everywhere but loses optimality as soon as there is any clutter: already at 5% obstacles only 35% of its trips are optimal, and at 30% only 7%; its paths were on average up to 20% longer.

Answer to the investigation

No algorithm is best for every map, but the choice is **not** symmetric. A* never lost on correctness and its cost is bounded above by UCS; greedy is faster but can be forced into long detours; UCS is only competitive when the heuristic carries no information (maze) – and even there A* is no worse.

The heuristic's value is a property of the **map**: large on open roads (20× fewer expansions than UCS), negligible inside mazes (1%).

1.8 Design decision – 95% open roads

To match the operations profile I generated 200 random trips on 40×40 maps with 5% obstacles (occasional detours) and averaged the measures:

Table 7. Operating profile: 200 trips, 5% obstacles.

Algorithm	Expanded	Generated	Max frontier	Optimal trips	Time (ms)
Uniform-cost	1325.6	1344.5	46.0	100.0%	3.02
Greedy best-first	50.4	133.3	83.8	44.5%	0.18
A*	60.7	151.4	83.0	100.0%	0.23

Recommendation: A* with the Manhattan heuristic (ties broken towards smaller h)

Why not UCS: it expands 1,326 nodes per trip against 61 for A* (≈ 22× more) for exactly the same, optimal answer – this is the penalty for ignoring the open-road structure.

Why not greedy: it is 20% cheaper than A* (50 vs 61 expansions, 0.18 vs 0.23 ms) but returns a non-optimal route on 55.5% of trips. A shuttle pays for distance on every trip, so a 4% longer route on most trips costs more than 0.05 ms of planning; and in the remaining ≈5% of trips with traps it can fail badly (17–46% longer routes in my maps).

Why A* is safe for the other 5%: in the trap and cluttered maps it stayed optimal and its effort stayed between greedy and UCS, so the worst case degrades gracefully towards UCS rather than towards wrong answers.

Condition that would change my decision: if planning time were truly the binding constraint (e.g. thousands of vehicles per second) greedy could be used as a first guess with A* repair; or weighted A* ($f = g + w \cdot h$) if bounded sub-optimality is acceptable.

Task 2 – When Is an Online Search Agent an Appropriate Choice? (4 marks)

2.1 Set-up and assumptions

- **Sensing:** the rover learns whether each of the 4 adjacent cells is open or a wall; it remembers everything it has seen. **The dock coordinates are known** (e.g. a beacon) but the walls are not; this is what lets LRTA* use a Manhattan heuristic.
- **Environments:** mazes generated by recursive back-tracking (perfect mazes: many dead ends, one route) and "braided" versions in which a chosen fraction of dead ends is opened to create loops. Dead-end density is measured as dead-end cells ÷ open cells.
- **Online DFS:** move to an unvisited open neighbour; if none, step back to the cell it came from; stop at the dock, or declare failure when back at the start with nothing unexplored.
- **Learning agent:** LRTA* keeps a table H(s) initialised to Manhattan distance; at each step moves to the neighbour minimising 1 + H(n) and then updates H(s) ← max(H(s), min 1 + H(n)). The table persists between trials, which is how learning happens.
- **Measures:** moves, revisited cells (moves that enter an already-visited cell) and competitive ratio = moves ÷ optimal cost with the full map (BFS).

2.2 Predictions made before running

- P1 – Competitive ratio is worst, and most variable, for **near** targets in large mazes: DFS explores in a fixed order that ignores the target, so a lucky or unlucky first turn dominates a small denominator.
- P2 – Ratio grows with maze size; removing dead ends (loops) reduces revisits.
- P3 – For an **unreachable** target online DFS terminates, but only after covering every reachable cell: moves ≈ 2 × (reachable cells − 1).
- P4 – LRTA* costs more than DFS on trial 1 and then improves over repeated trials towards the optimum; on an unreachable target it never terminates.

2.3 Experiment A – target distance

60 random 31×31 perfect mazes; the target is the cell at the 10th, 30th, 50th, 70th, 90th and 100th percentile of optimal distance from the start.

Table 8. Online DFS versus target distance (31×31, 60 mazes).

Target distance (percentile)	Mean optimal cost	Mean DFS moves	Mean revisits	Mean ratio	Std dev	Worst ratio
10%	41.3	75.5	17.1	1.82	3.40	20.3
30%	105.7	216.2	55.3	2.07	0.95	7.1
50%	157.2	430.9	136.9	2.79	1.37	5.3
70%	203.4	407.4	102.0	2.07	0.91	4.6
90%	246.9	454.8	103.9	1.88	0.45	3.0
100%	271.7	492.2	110.3	1.84	0.40	3.0

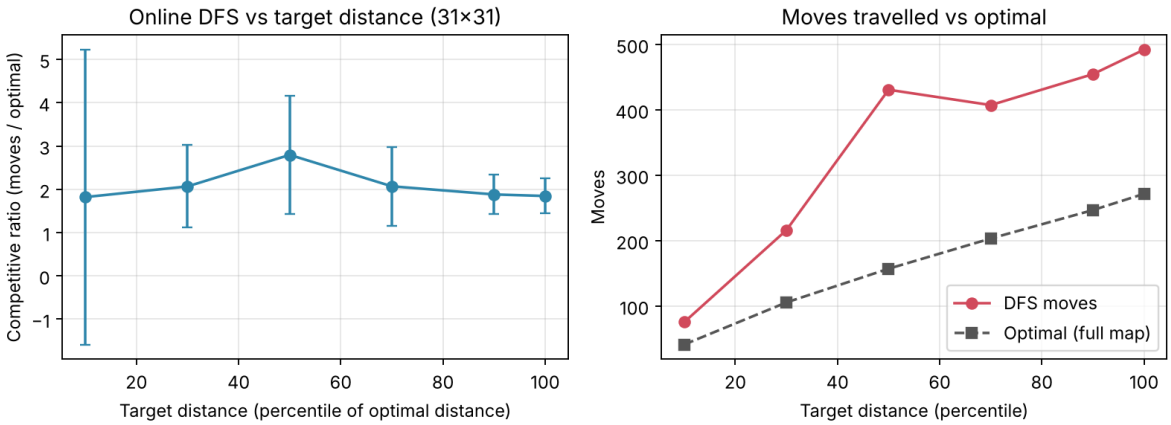


Figure 4. Competitive ratio (mean ± one standard deviation) and moves travelled against optimal moves as the target moves further away.

Finding. P1 is only partly right. The **worst** case is clearly the near target (one run needed 20.3× the optimal moves and the standard deviation is 3.4, against 0.4 for the farthest target), but the **mean** ratio is highest for mid-distance targets (2.79) and about 1.8 for near and far targets. Reason: DFS wastes a roughly fixed amount of exploration regardless of the target, which matters most when the optimal cost is tiny; for far targets the exploration is mostly "on the way".

2.4 Experiment B – maze size and dead-end density

30 mazes per cell, start (1,1), target in the opposite corner.

Table 9. Online DFS to the far corner: effect of size and loops.

Size	Dead ends removed	Dead-end density	Open cells	Mean moves	Mean revisits	Mean ratio	Worst ratio
11×11	0%	7.7%	49	50	11	1.97	4.0
11×11	50%	4.3%	51	43	8	2.20	5.0
11×11	100%	0.0%	53	42	7	2.58	5.3
21×21	0%	5.9%	199	240	76	3.03	5.6
21×21	50%	2.9%	204	185	49	3.28	8.0
21×21	100%	0.0%	210	233	76	5.18	9.4
31×31	0%	5.4%	449	611	216	3.95	8.7
31×31	50%	2.6%	460	404	112	4.22	8.1
31×31	100%	0.0%	473	510	166	6.84	12.5
41×41	0%	5.3%	799	1,193	456	5.10	12.3
41×41	50%	2.6%	820	975	349	7.64	15.7
41×41	100%	0.0%	840	885	285	8.07	13.4
61×61	0%	5.1%	1,799	2,560	958	4.55	10.0
61×61	50%	2.5%	1,845	2,097	761	9.85	18.0
61×61	100%	0.0%	1,891	1,983	668	12.83	23.6

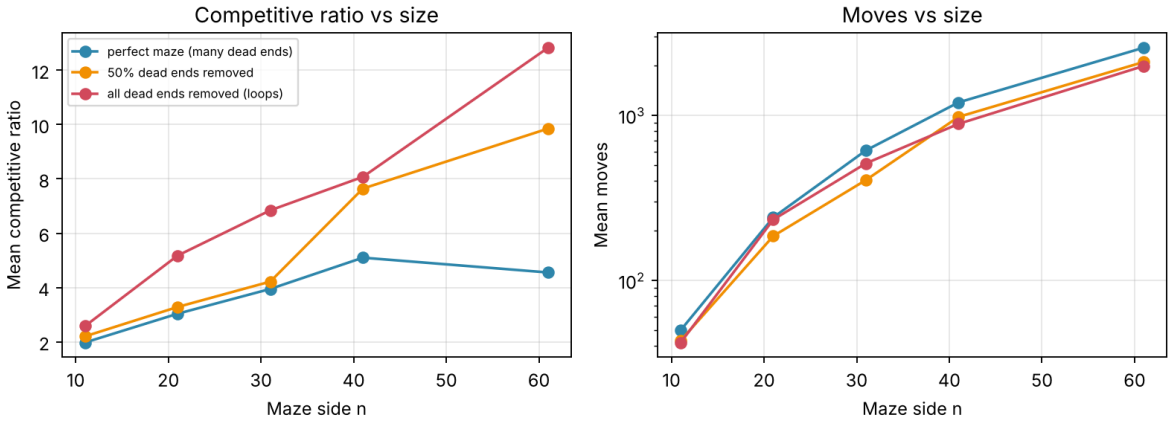


Figure 5. Competitive ratio and moves against maze size for three dead-end densities.

Finding. Moves grow roughly linearly with the number of cells (about 1.0–1.4 moves per cell), but the **competitive ratio keeps rising with size**: from 2.0 (11×11) to 5.1 (41×41) in perfect mazes, and to 12.8 (61×61) when all dead ends are removed. P2 is half right: loops do cut revisits (e.g. 41×41: 456 → 285) but they also shorten the optimal path, which shrinks the denominator faster, so the **ratio gets worse, not better**, with loops (8.1 against 5.1 at 41×41).

2.5 Experiment C – unreachable target

The dock is placed in a cell sealed off by walls. 30 mazes per row.

Table 10. Online DFS on an unreachable target.

Size	Dead ends removed	Reachable cells	Mean moves	Moves per reachable cell	Mean revisits
11×11	0%	49	96	1.96	48
11×11	100%	53	104	1.96	52
21×21	0%	199	396	1.99	198
21×21	100%	211	419	1.99	210
31×31	0%	449	896	2.00	448
31×31	100%	474	945	2.00	473
41×41	0%	799	1,596	2.00	798
41×41	100%	841	1,679	2.00	840

Finding (P3 confirmed). DFS terminates and reports failure after about 2 moves per reachable cell (1.96–2.00), exactly the cost of walking every corridor and back. There is no competitive ratio because the optimal cost is undefined; the cost of *proving* "unreachable" grows linearly with the facility. LRTA* is not complete here: on a 21×21 maze with an unreachable dock it did not terminate within 50,000 moves, while DFS stopped after 396 moves (199 reachable cells).

2.6 Experiment D – learning across repeated missions (LRTA*)

30 mazes of 21×21, target in the far corner, 10 consecutive trials with the H table kept between trials.

Table 11. LRTA* moves per trial on the same maze.

Trial	Perfect maze: mean moves	Ratio to optimal	Loopy maze: mean moves	Ratio to optimal
1	455.5	4.99	210.3	4.60
2	325.6	3.57	172.1	3.76
3	271.0	2.97	155.1	3.39

Trial	Perfect maze: mean moves	Ratio to optimal	Loopy maze: mean moves	Ratio to optimal
4	258.7	2.84	90.1	1.97
5	244.4	2.68	86.7	1.90
6	208.3	2.28	98.2	2.15
7	198.5	2.18	93.0	2.03
8	187.5	2.06	82.0	1.79
9	198.1	2.17	74.3	1.63
10	159.3	1.75	72.7	1.59
Optimal	**91.2**	**1.00**	**45.7**	**1.00**
Online DFS (one mission)	**228.9**	**2.51**	**211.7**	**4.63**

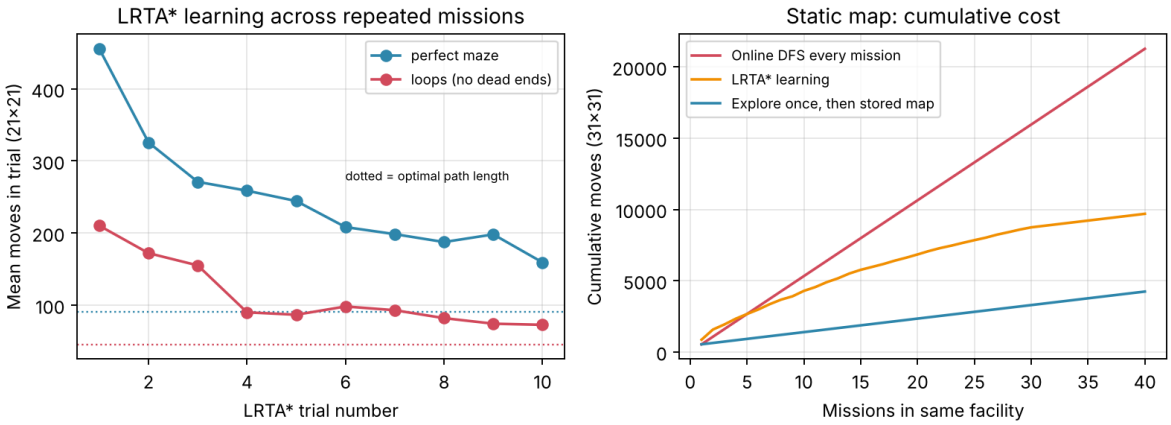


Figure 6. Left: LRTA* learning curve (dotted lines = optimal). Right: cumulative moves over repeated missions in one 31×31 facility for three strategies.

Finding (P4 confirmed). Trial 1 of LRTA* is worse than DFS in the perfect maze (456 vs 229 moves) because the Manhattan heuristic lures it into dead ends that it must fill with learned values. It then improves (159 moves by trial 10) but is **still 1.7× optimal** after ten missions: LRTA* converges to the optimum slowly.

2.7 Answer: when is an online agent a reasonable choice?

Operating boundary (evidence-based)

Use an online agent when the map is genuinely unavailable **and** the mission is rare or one-off **and** the facility is small. My acceptance criterion is a competitive ratio of at most ≈ 3 (the rover's battery covers three times the shortest route). In perfect mazes the mean ratio reaches 3.0 at 21×21 (≈ 200 cells) and 3.9 at 31×31 (≈ 450 cells); with loops it already exceeds 5 at 21×21. So the boundary is **roughly 200–500 reachable cells**, and lower in loop-rich buildings.

Stop recommending it when: (i) the facility exceeds ≈ 500 cells (ratio ≥ 4 and 600+ moves for a single trip; 61×61 loopy: ratio 12.8); (ii) the facility will be visited **more than once** with an unchanged layout, since DFS pays the exploration cost every time while LRTA* needs ≳ 10 missions to approach 1.7× optimal; (iii) failure must be certified cheaply, because proving "unreachable" costs ≈ 2 moves per cell.

Repeated missions: online search is a mapping tool, not a navigation strategy. From the second mission on, the cost of rediscovery is pure waste.

2.8 Changed condition – 20 rovers, 20 facilities, maps unchanged for months

Table 12. Cumulative moves in one 31×31 facility (30 mazes, 50% dead ends removed): mean DFS mission 532 moves, optimal route 95 moves.

Missions per facility	DFS every mission	LRTA* learning	Explore once, then stored map
1	532	853	532
2	1,064	1,567	627
5	2,659	2,659	912
10	5,319	4,280	1,386
20	10,637	6,838	2,336
40	21,275	9,690	4,234

Re-evaluation. With stable maps the answer changes: the first mission in each facility should use **online DFS purely to build and store the map**; all later missions should plan offline on the stored map with A* and drive the optimal 95-move route instead of 532 moves. After only two missions the stored-map strategy is already ahead (627 vs 1,064). For 20 facilities × 40 missions each the totals are **425,493 moves** (DFS every time), **193,807** (LRTA* learning) and **84,685** (map once, reuse) – a saving of 80.1% against re-exploring.

LRTA* is the weakest of the three here: it only breaks even with repeated DFS after ≈ 5 missions and is still 2.3× more expensive than map reuse after 40. Because the facilities differ, the 20 rovers cannot share maps, so the one-time DFS cost is paid per facility – but it is paid once. A small safeguard is needed because “unchanged for months” is not “unchangeable”: if a stored-open cell is found blocked, the rover falls back to online search locally and updates the map.

Task 3 – Engineering the Behaviour of Alpha-Beta Pruning (6 marks)

3.1 Method

Explicit game trees with branching factor b = 4 and depth d = 6 (4,096 leaves, 5,461 nodes in total). The root is a MAX node. Plain minimax and fail-soft alpha-beta (cut-off test $\alpha \geq \beta$) are run on the **same tree**; both must return the same root value (asserted in the code). Counted: nodes visited (every call, leaves included), leaf evaluations and cut-offs (number of times a loop was abandoned early).

Three tree conditions were designed, plus one extra:

- **Best ordering (many cut-offs):** random leaf values, then at every node the children are sorted by their true minimax value – best child first for MAX (highest) and for MIN (lowest).
- **Engineered worst case (almost none/no cut-offs):** sorting in reverse is *not* enough (see 3.4), so leaf values are generated top-down with a nested-window rule. A MAX node with window (lo, hi) gets children whose values strictly increase, each child receiving the window (previous best, hi); a MIN node gets strictly decreasing values with window (lo, previous best); a leaf takes the midpoint of its window. Every value therefore lies strictly inside the (α, β) window alpha-beta holds on arrival, so $\alpha \geq \beta$ never occurs.
- **Random ordering:** 200 trees with random leaf values and children in random order.
- **All leaf values equal** (the additional condition of my choice).

3.2 Predictions made before execution

For a perfectly ordered tree alpha-beta must only *refute* the non-best children, so the visited tree is the minimal tree: level i contains $b^{\lceil i/2 \rceil} + b^{\lfloor i/2 \rfloor} - 1$ nodes. Leaves: $4^3 + 4^3 - 1 = 127$. Summing the levels (1, 4, 7, 19, 31, 79, 127) gives 268 nodes. With the worst ordering no bound ever crosses, so every node is visited: 4,096 leaves and 5,461 nodes, 0 cut-offs. For random order I used the asymptotic rule of thumb $b^{(3d/4)} = 4^{4.5} \approx 512$ leaves. With all values equal, ties satisfy $\alpha \geq \beta$, so I expected behaviour similar to the best case (127–300 leaves).

3.3 Results

Table 13. Alpha-beta on b = 4, d = 6. Minimax visits 5,461 nodes / 4,096 leaves in every case.

Tree	Predicted leaves / nodes	Observed leaves / nodes (alpha-beta)	Cut-offs	Comparison
Best ordering	127 / 268	127 / 268	99	Exact match
Engineered worst	4,096 / 5,461	4,096 / 5,461	0	Exact match
Random order (mean of 200)	≈ 512 / –	867 / 1,350	302	Under-estimated by 1.7× (range 421–1291)
All leaves equal	127–300 / –	127 / 268	99	Within range (identical to best case)
Reverse-sorted random tree (extra)	4,096 / 5,461	3,568 / 4,853	709	**Wrong**: 13% of leaves were still pruned

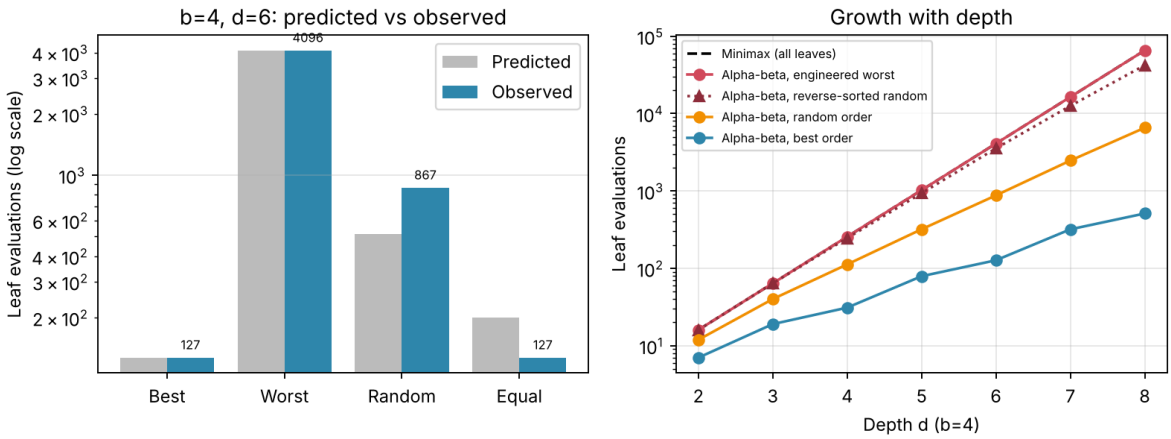


Figure 7. Left: predicted and observed leaf evaluations (log scale). Right: growth of leaf evaluations with depth for b = 4.

Table 14. Leaf evaluations against depth (b = 4).

Depth	Minimax leaves	Engineered worst	Reverse-sorted random	Random order (mean)	Best ordering	Best formula
2	16	16	16	12	7	7
3	64	64	64	40	19	19
4	256	256	245	112	31	31
5	1,024	1,024	934	319	79	79
6	4,096	4,096	3,548	876	127	127
7	16,384	16,384	12,571	2,469	319	319
8	65,536	65,536	42,081	6,552	511	511

3.4 Comparing prediction with experiment

- **Best case matched exactly** (127 leaves, 268 nodes) and so did the closed form at every depth in the sweep (7, 19, 31, 79, 127, 319, 511 leaves for d = 2…8, including the odd-depth cases).
- **Engineered worst case matched exactly**: 0 cut-offs, alpha-beta = minimax. My first attempt at a “worst” tree, simply reversing the best ordering of random values, **failed my prediction**: 709 cut-offs and 3,568 leaves (87% of the tree). Reversing the order of siblings does not control the *bounds*: values deep inside a subtree still fall outside the window inherited from an ancestor and are cut. A true no-cut-off tree needs values that are mutually consistent across levels, which is what the nested-window construction provides.
- **Random order:** the prediction was low by a constant factor (observed 867 vs 512), but the growth per extra ply (×2.65–2.8 at d = 6→8) matches b^(3/4) = 2.83. The rule of thumb gives the *growth rate*, not the constant.
- **All values equal:** exactly the best-case numbers (127 / 268 / 99 cut-offs). Every sibling after the first is cut because its value equals the bound (α ≥ β holds with equality). With a strict test (α > β) there would be no cut-offs: the exact form of the test is part of the algorithm’s cost.

3.5 Inquiry

Why does one tree need substantially more work than another?

Alpha-beta can skip a branch only when it already holds a bound that proves the branch cannot change the result. In the best tree the very first child at each node produces the strongest bound (a high α at MAX nodes, a low β at MIN nodes), so the other children of a node are examined only until they are refuted – typically one child each. In the worst tree the best child comes last, so every bound arrives too late: by the time a refutation

exists, the work it could have prevented has been done. Same tree, same value, 127 versus 4,096 leaf evaluations (32×).

Which characteristics of move ordering create or prevent cut-offs?

- **Creates:** at MAX nodes the highest-valued child is searched first; at MIN nodes the lowest-valued child first; this ordering applied consistently at all levels (not just the root). Ties also create cut-offs under $\alpha \geq \beta$.
- **Prevents:** the opposite order (best last) at any level, and values arranged so every later sibling improves on the previous bound (strictly monotone values in the nested-window tree).
- **Partial order** gives partial savings: random order sits between the extremes (867 leaves, i.e. 21% of the tree) and the saving comes from sometimes searching a good child early by chance.

What does this reveal about judging an algorithm by b and d alone?

Table 15. Five (b, d) shapes with the same number of leaves (4,096).

b	d	Leaves of full tree (b^d)	Alpha-beta best	Alpha-beta random (mean of 30)	Alpha-beta worst
2	12	4,096	127	814	4,096
4	6	4,096	127	878	4,096
8	4	4,096	127	869	4,096
16	3	4,096	271	1,176	4,096
64	2	4,096	127	1,118	4,096

The pair (b, d) fixes the size of the minimax tree but **not** the work of alpha-beta: for one and the same tree shape the cost spans a factor of 32 (127 to 4,096 leaves) depending on ordering. In the best case the cost depends only on $b^{(d/2)} \approx \sqrt{(b^d)}$ (127 for four different shapes, 271 for the odd depth d = 3), while in the worst case it is b^d. Complexity statements such as $O(b^{(d/2)})$ and $O(b^d)$ describe two different *inputs*, and the average case $b^{(3d/4)}$ lies in between and depends on the quality of ordering. A fair evaluation must therefore report the ordering assumption (or measure on realistic ordered trees), cut-off counts and node counts, not just b and d.

Task 4 – Minimax and the Cost of Examining Every Possibility (6 marks)

4.1 Game and evaluation function

The game is an **N×N k-in-a-row game** (generalised tic-tac-toe): players X (maximiser) and O (minimiser) alternately fill empty cells; the first to complete k in a row, column or diagonal wins. Four board settings are used: 3×3 (k=3), 4×4 (k=4), 5×5 (k=4), 6×6 (k=4). The branching factor is not constant: it is the number of empty cells c, which falls by 1 each ply.

My evaluation function E (from X's viewpoint). Consider every line of k consecutive cells. If it contains only X stones (x of them) it scores +W[x]; if it contains only O stones (o of them) it scores −W[o]; a mixed or empty line scores 0, because it can no longer be won by either side. W = 0, 1, 10, 100, 1000… (each extra stone in an unblocked line is worth ten times more). E is the sum over all lines. A completed line is a terminal win scored ±(100000 + remaining depth), so that faster wins and slower losses are preferred. E is maintained incrementally (only the lines through the changed cell are re-scored); a test compares it with the full recomputation on 200 random positions.

4.2 Derivation before implementation

Let c be the number of empty cells at the root and d the lookahead depth, and assume no terminal position is reachable within d plies.

- **Positions at ply i:** P(c,i) = c·(c−1)⋯(c−i+1) = c!/(c−i)! (each ply removes one choice).
- **Total nodes visited:** N(c,d) = Σ_{i=0..d} c!/(c−i)!. Internal (expanded) nodes: Σ_{i=0..d−1} c!/(c−i)!.
- **Evaluation-function calls:** nodes at the depth limit, P(c,d) = c!/(c−d)!. With constant branching factor b this reduces to b^d leaves and N = (b^(d+1) − 1)/(b − 1) ≈ b^d·b/(b−1).
- **Change with d:** leaf evaluations are multiplied by (c − d + 1) when d increases by one, i.e. ≈ ×c at first and slowly less. Leaves dominate: the leaf share is P(c,d)/N(c,d) = 1 − 1/(c−d+1) approximately, e.g. 92–95% for the boards used.
- **Use of E at cut-off:** when depth reaches 0 the search does not look further; E replaces the unknown game-theoretic value. A win/loss/draw found earlier is returned directly (terminal evaluation, no call to E).

Depth-limited minimax (pseudocode of game.py / task4.py)

```
def minimax(depth, player):                  # value from X's viewpoint
    nodes += 1
    if depth == 0:  evals += 1; return E(position)          # cut-off
    best = -inf if player == X else +inf
    for each empty cell m:
        if make(m, player) completes a line: v = player * (100000 + depth)  # terminal
        else:                                v = minimax(depth-1, -player)
        unmake(m); best = max(best, v) if player == X else min(best, v)
    return best
```

4.3 Experimental results – growth

Search from the **empty board** at each depth (one run; counts are deterministic). "Formula" is N(c,d) derived above.

Table 16. Observed node and evaluation counts against the derived formulas.

Board	Depth	Nodes (observed)	Nodes (formula)	Evaluation calls	Formula P(c,d)	Time (s)	Match
3x3 (k=3)	1	10	10	9	9	<0.01	Exact
3x3 (k=3)	2	82	82	72	72	<0.01	Exact

Board	Depth	Nodes (observed)	Nodes (formula)	Evaluation calls	Formula P(c,d)	Time (s)	Match
3x3 (k=3)	3	586	586	504	504	<0.01	Exact
3x3 (k=3)	4	3,610	3,610	3,024	3,024	<0.01	Exact
3x3 (k=3)	9	549,946	986,410	46,080	362,880	0.81	Terminal states
4x4 (k=4)	1	17	17	16	16	<0.01	Exact
4x4 (k=4)	2	257	257	240	240	<0.01	Exact
4x4 (k=4)	3	3,617	3,617	3,360	3,360	<0.01	Exact
4x4 (k=4)	4	47,297	47,297	43,680	43,680	0.04	Exact
4x4 (k=4)	5	571,457	571,457	524,160	524,160	0.54	Exact
4x4 (k=4)	6	6,337,217	6,337,217	5,765,760	5,765,760	6.25	Exact
5x5 (k=4)	1	26	26	25	25	<0.01	Exact
5x5 (k=4)	2	626	626	600	600	<0.01	Exact
5x5 (k=4)	3	14,426	14,426	13,800	13,800	0.02	Exact
5x5 (k=4)	4	318,026	318,026	303,600	303,600	0.48	Exact
5x5 (k=4)	5	6,693,626	6,693,626	6,375,600	6,375,600	8.42	Exact
6x6 (k=4)	1	37	37	36	36	<0.01	Exact
6x6 (k=4)	2	1,297	1,297	1,260	1,260	<0.01	Exact
6x6 (k=4)	3	44,137	44,137	42,840	42,840	0.06	Exact
6x6 (k=4)	4	1,457,857	1,457,857	1,413,720	1,413,720	2.19	Exact

For **every** configuration in which no terminal position can occur within the horizon (19 of 20 rows) the counts equal the formulas **exactly**. The 3×3 depth-9 search (full game) is the exception: wins end branches early, so only 549,946 of the 986,410 formula nodes are visited and the formula is an upper bound.

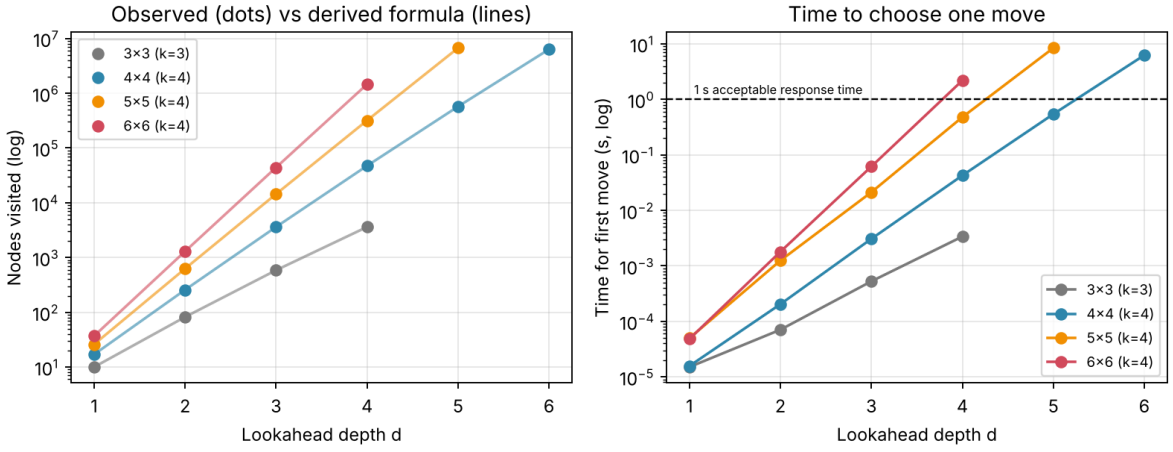


Figure 8. Left: observed nodes (dots) lie on the derived curves. Right: time to choose the first move against depth; the dashed line marks the 1-second criterion.

Throughput was ≈ 0.7–1.1 million nodes per second in this Python implementation, nearly constant, so **time is proportional to nodes** (an operation count remains the primary evidence; time is shown as supporting evidence).

4.4 Quality of the chosen move

For each setting the minimax agent played 20 games against a random player (10 as first player, 10 as second).

Table 17. Minimax agent versus a random player, 20 games each.

Board	Depth	Wins	Draws	Losses	Win rate	Mean nodes per game
3x3 (k=3)	1	19	1	0	95.0%	19

Board	Depth	Wins	Draws	Losses	Win rate	Mean nodes per game
3x3 (k=3)	2	18	2	0	90.0%	133
3x3 (k=3)	3	20	0	0	100.0%	744
3x3 (k=3)	4	18	2	0	90.0%	3,625
3x3 (k=3)	9 (full)	16	4	0	80.0%	309,569
4x4 (k=4)	1	20	0	0	100.0%	51
4x4 (k=4)	2	17	3	0	85.0%	707
4x4 (k=4)	3	15	5	0	75.0%	7,806
4x4 (k=4)	4	14	6	0	70.0%	81,402
5x5 (k=4)	1	20	0	0	100.0%	85
5x5 (k=4)	2	20	0	0	100.0%	1,995
5x5 (k=4)	3	20	0	0	100.0%	38,593

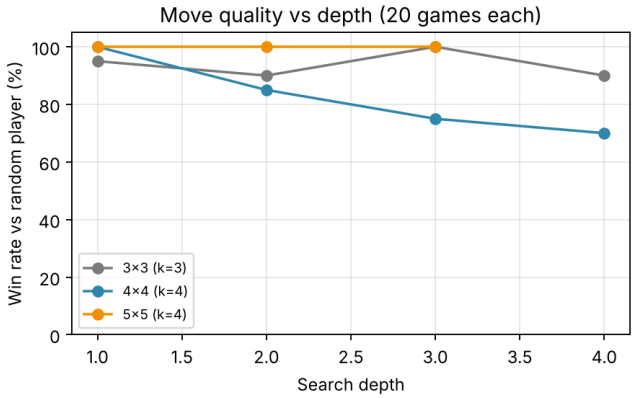


Figure 9. Win rate against a random opponent by depth.

Interpretation. The agent **never lost** to a random player at any depth, but the win rate does not rise with depth: it is 100% at depth 1 on 4×4 and falls to 70% at depth 4, with draws taking the difference. A random player is too weak to separate depths: a deeper search sees that many moves lead to the same game-theoretic outcome and plays safe, drawn lines rather than racing for a quick win. To measure depth itself I added head-to-head matches (20 games, colours alternating):

Table 18. Depth versus depth (20 games each).

Board	Deeper agent	Shallower agent	Wins	Draws	Losses
4x4 (k=4)	depth 2	depth 1	0	20	0
4x4 (k=4)	depth 3	depth 1	0	20	0
4x4 (k=4)	depth 3	depth 2	0	20	0
4x4 (k=4)	depth 4	depth 2	0	20	0
5x5 (k=4)	depth 2	depth 1	2	15	3
5x5 (k=4)	depth 3	depth 1	7	6	7
5x5 (k=4)	depth 3	depth 2	8	8	4

On 4×4 (k=4), a drawish game, every match was drawn. On 5×5, depth 3 beat depth 2 (8 wins, 4 losses) but was level with depth 1 (7–7): with this simple evaluation function extra depth gives a modest, not guaranteed, strength gain. The quality of E, not only depth, limits playing strength.

4.5 Scalability inquiry

> **Acceptable response time**
>
> I define acceptable as **at most 1 second to choose a move** in this experimental environment (an interactive turn-based game: a human should not wait noticeably).

Table 19. Practical depth limit of plain minimax (empty board).

Board	Deepest plain-minimax depth within 1 s	Time at that depth	Next depth: time	Leaves at next depth
3×3 (k=3)	Full game (depth 9)	0.81 s	–	–
4×4 (k=4)	5	0.54 s	6: 6.25 s	5,765,760
5×5 (k=4)	4	0.48 s	5: 8.42 s	6,375,600
6×6 (k=4)	3	0.06 s	4: 2.19 s	1,413,720

When plain minimax becomes impractical. With ≈ 10^6 nodes per second, the 1-second limit corresponds to about 10^6 nodes, i.e. when $c\cdot(c-1)\cdots(c-d+1)$ exceeds ≈ 10^6. This happens at depth 6 on 16 cells, depth 5 on 25 cells and depth 4 on 36 cells. Each additional ply multiplies the cost by roughly 10–35 (the number of remaining empty cells), so a 10× faster computer buys less than **one** more ply. A 7×7 board (49 cells) would already exceed 5×10^6 leaves at depth 4.

Where the inefficiency originates (measured in Task 7): (1) 92–95% of all nodes are leaf evaluations; (2) the same position is reached through different orders of the same moves and re-evaluated each time; (3) all siblings are expanded even when the decision is already settled by a better or refuting move. **A redesigned strategy would need to** avoid re-searching positions already seen, abandon branches that can no longer change the choice, and examine promising moves first so that the second saving actually happens. It would also have to keep returning the *same* move value as minimax. I do not implement it here; Task 7 does.

Task 5 – Deriving a Constraint-Propagation Algorithm from a Principle (7 marks)

5.1 From the principle to a procedure

Given principle: *a value may be removed from a variable's domain if some neighbouring variable has no value compatible with it.* I converted it into a procedure by answering the five design questions.

Table 20. Derivation of the procedure.

Question	My design decision
How are the constraints represented?	Variables are exams, domains D(Xi) are sets of slots. Two exams that share a student are joined by an edge of a conflict graph (stored as adjacency lists). Each edge is a binary constraint C(i,j)(v,w) ≡ v ≠ w. Support is directional, so every edge gives two **arcs** (i→j) and (j→i).
How is an unsupported value detected?	v ∈ D(Xi) is unsupported with respect to neighbour Xj if no w ∈ D(Xj) satisfies C(v,w). REVISE(Xi, Xj) tests each v against the values of D(Xj) and stops at the first support; each compatibility test counts as one constraint check.
Which pairs must be re-examined after a removal?	A removal from D(Xi) can only destroy the support of values in neighbours **of Xi**. So after REVISE(Xi, Xj) removes something, the arcs (Xk → Xi) for every neighbour Xk ≠ Xj are queued (the arc (Xj → Xi) need not be, since Xj was the cause). Nothing else can be affected.
What indicates no solution?	A **domain wipe-out**: D(Xi) becomes empty. A solution would have to give Xi a value from its domain, so none exists.
How is it combined with backtracking?	After choosing Xi = v the domain is replaced by {v} and the arcs (Xk → Xi) of its neighbours are propagated. A wipe-out means this choice is refuted immediately and the next value is tried; otherwise the search recurses on the reduced domains (which are copied so they can be restored). This is "maintaining arc consistency" (MAC).

My pseudocode (written before implementation)

```
PROPAGATE(domains, queue):
    while queue not empty:
        (Xi, Xj) <- pop(queue)
        if REVISE(Xi, Xj):
            if domains[Xi] is empty: return FAILURE            # wipe-out
            for Xk in neighbours(Xi), Xk != Xj: push (Xk, Xi)  # re-examine
    return OK

REVISE(Xi, Xj):  removed <- false
    for v in domains[Xi]:
        if no w in domains[Xj] with w != v:                    # unsupported
            remove v from domains[Xi]; removed <- true
    return removed

SOLVE(domains, assigned):
    if all assigned: return solution
    X <- choose unassigned variable
    for v in domains[X]:
        D' <- copy(domains); D'[X] <- {v}
        if PROPAGATE(D', [(Xk, X) for Xk in neighbours(X)]) == OK:
            r <- SOLVE(D', assigned + {X})
            if r != failure: return r
    return failure
```

Two search variants are compared: **BT** is plain backtracking in static (index) order that checks each new value only against already-assigned neighbours; **MAC** is the procedure above, with the root propagated first; **MAC+MRV** additionally picks the unassigned variable with the smallest domain (ties: most neighbours). Counters: *nodes* = value assignments tried; *backtracks* = assignments undone; *constraint checks* = compatibility tests; *wipe-outs* = empty domains found.

5.2 Test instance 1 – propagation alone decides

Six exams in a conflict chain E0–E1–E2–E3–E4–E5 with three slots. Initial domains: E0 {S1}, E1 {S1,S2}, E2 {S2,S3}, E3 {S1,S3}, E4 {S1,S2}, E5 {S2,S3}.

Table 21. Trace of the propagation on instance 1.

Step	Arc revised	Value removed	Because
1	E1 → E0	S1 from D(E1)	E0 has no slot different from S1
2	E2 → E1	S2 from D(E2)	E1 has no slot different from S2
3	E3 → E2	S3 from D(E3)	E2 has no slot different from S3
4	E4 → E3	S1 from D(E4)	E3 has no slot different from S1
5	E5 → E4	S2 from D(E5)	E4 has no slot different from S2

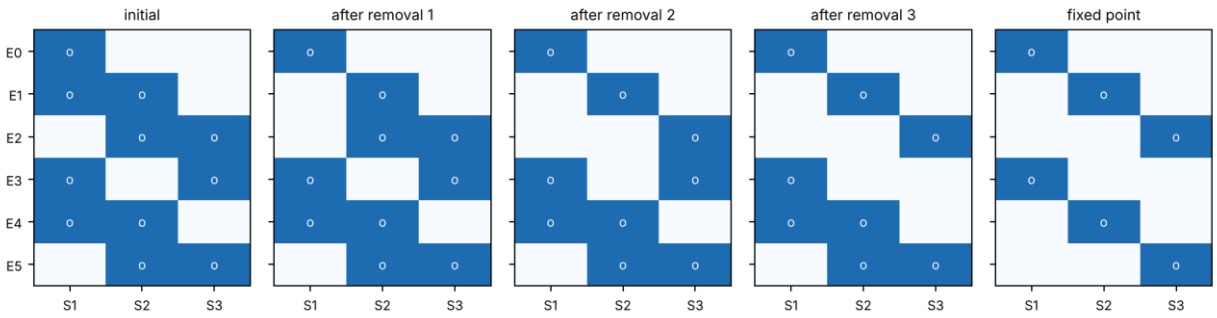


Figure 10. Domains of the six exams (rows) over slots S1–S3 (columns); a mark means the slot is still allowed. The propagation shrinks the domains to singletons.

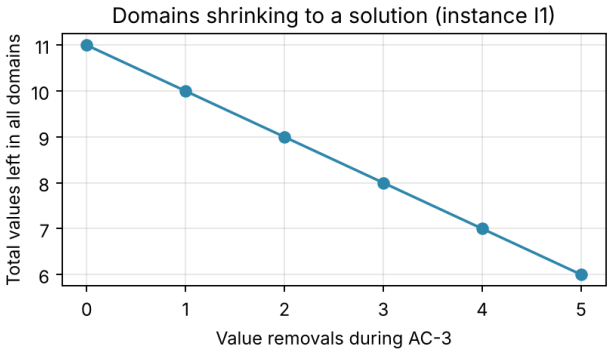


Figure 11. Total number of values left in all domains after each removal (11 → 6).

The fixed point is E0=S1, E1=S2, E2=S3, E3=S1, E4=S2, E5=S3: every domain is a single value, so the solution is **read off with no search decision** (5 removals, 23 constraint checks in the first pass). Changing the domain of E5 to {S2} gives instance 1b: the same cascade forces E1=S2, E2=S3, E3=S1 and then E4=S2, but E5 can only take S2, so S2 is removed from D(E4) as unsupported and D(E4) becomes empty. The instance is **proved infeasible by propagation, with 0 search nodes** (MAC: 16 checks, 1 wipe-out; plain BT needed 5 nodes and 5 backtracks to find the same conclusion).

Table 22. Instances 1 and 1b. On such tiny instances plain BT is also cheap; the point is that propagation settles them without search.

Instance	Method	Nodes	Backtracks	Constraint checks	Wipe-outs	Result
I1	BT	6	0	9	0	Solved
I1	MAC	6	0	33	0	Solved (0 choice points)
I1b	BT	5	5	9	0	Infeasible
I1b	MAC	0	0	16	1	Infeasible (root wipe-out)

5.3 Test instance 2 – designed to challenge the algorithm

Instance I2: four exams that all conflict with each other (a K4 clique) but only **three slots**, placed **last** behind nine loosely constrained exams (13 exams in total, every domain {S1,S2,S3}). It is infeasible by the pigeonhole principle. My propagation sees nothing wrong: every value of every exam still has a support in every neighbour, so **0 values are removed** and no wipe-out occurs.

Table 23. Instance 2 (adversarial).

Method	Nodes	Backtracks	Constraint checks	Wipe-outs	Result
BT (static order)	9,213	9,213	42,999	0	Infeasible
MAC (static order)	6,141	6,141	83,037	3,072	Infeasible
MAC + MRV	9	9	264	6	Infeasible

What it shows. (i) Arc consistency is **incomplete**: pairwise support does not capture "four exams need four different slots". (ii) With a static order MAC is barely better in nodes (6,141 vs 9,213) and **worse in constraint checks** (83,037 vs 42,999) because it pays for propagation at each node without gaining pruning: the algorithm thrashes through every consistent assignment of the nine harmless exams before reaching the clique. (iii) The cure is not stronger propagation but a **fail-first variable order** (MRV with degree tie-break): the clique is tackled first and the failure is found with 9 nodes and 264 checks.

5.4 Backtracking with and without propagation – growth with the number of exams

Random timetables: n exams, 5 slots, each exam allowed in 3 randomly chosen slots, conflict graph with average degree 4.6; 25 instances per n (the same instances for every method); BT/MAC stopped after 200,000 assignments ("capped").

Table 24. Mean work per instance. BT figures for n ≥ 25 under-state the true cost because capped runs are counted at the cap.

n	BT nodes	MAC nodes	MAC+MRV nodes	BT checks	MAC checks	MAC+MRV checks	BT backtracks	MAC wipe-outs	BT capped
10	26	11	10	156	392	374	16	1	0/25
15	66	16	15	465	552	546	51	1	0/25
20	524	21	20	4,187	751	738	504	1	0/25
25	18,110	25	25	132,549	879	874	18,085	0	1/25
30	16,768	31	30	141,825	1,031	1,042	16,738	0	2/25
35	45,153	37	35	373,403	1,315	1,284	45,119	1	5/25
40	97,610	77	40	819,433	2,438	1,504	97,573	19	10/25

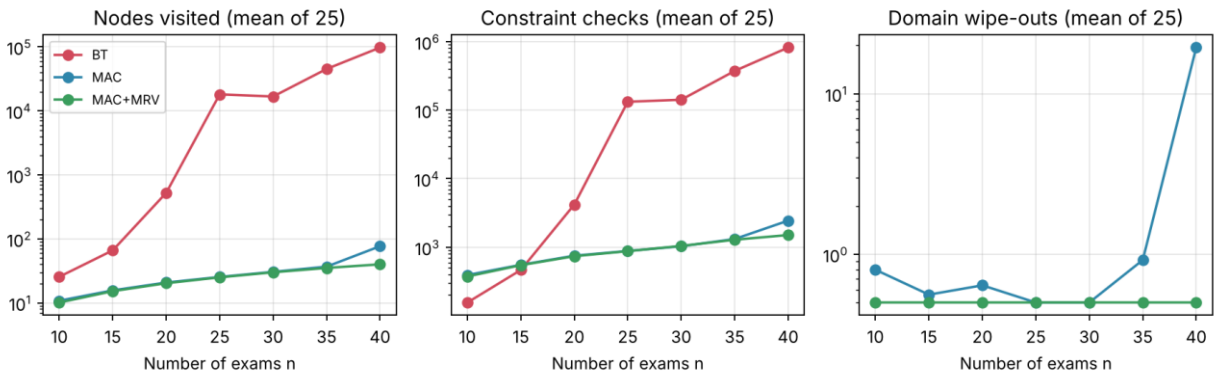


Figure 12. Mean nodes, constraint checks and wipe-outs against the number of exams (log scale).

- **Backtracking alone grows exponentially:** from 26 nodes (n=10) to 524 (n=20) to ≈ 98,000 (n=40), and 10 of 25 instances at n=40 were abandoned at the cap (only 15 finished).
- **With propagation the search is almost backtrack-free:** MAC visits ≈ n nodes (11 → 77) and MAC+MRV exactly n (n=40: 40 nodes, 1,504 checks) – each assignment propagates until nothing is left to choose. Work grows roughly linearly in n instead of exponentially.
- **Propagation has a price:** at n ≤ 15 MAC performs **more** constraint checks than plain BT (392 vs 156 at n=10) because every assignment triggers a propagation pass; the break-even is near n = 20 (751 vs 4,187), after which MAC is cheaper by one to three orders of magnitude.
- **Wipe-outs** are the mechanism: they are the early failures that replace deep backtracking.

5.5 Correctness inquiry – why removing an unsupported value is safe

Argument (by reasoning, not by output)

Claim. If v is removed from D(Xi) because some neighbour Xj has no value compatible with v, then no solution of the problem assigns Xi = v. Hence no value belonging to a solution is ever removed.

Invariant. At every moment, every solution S satisfies S(X) ∈ D(X) for every variable X. *Initially* this is true because domains are the allowed slots. *Step:* suppose it holds and v is about to be removed from D(Xi) because no w ∈ D(Xj) satisfies C(v,w). Assume, for contradiction, that some solution S has S(Xi) = v. S satisfies the constraint between Xi and Xj, so w* = S(Xj) is compatible with v; by the invariant w* ∈ D(Xj). Then w* would be a support for v – contradicting the assumption that no support exists. Hence S(Xi) ≠ v for every solution, and the removal preserves the invariant.

Consequences. (1) Propagation is *sound*: when it stops, all solutions are still inside the reduced domains. (2) An empty domain proves there is no solution, since a solution would need a value from it. (3) In MAC the same argument applies to the sub-problem "all solutions extending the current assignment", because setting D(X) = {v} just restricts which solutions are considered.

What it does not claim. The converse is false: non-empty arc-consistent domains do not guarantee a solution (instance I2). Propagation also terminates, since each iteration either removes a value or shortens the finite queue.

Task 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 marks)

6.1 Design of the exhaustive method

Table 25. Exhaustive search design.

Aspect	Design
Representation	Depot = city 0 (fixed start and end). A candidate tour is an ordered list of the n customers 1…n; distances are Euclidean, stored in an (n+1)×(n+1) matrix.
Generation	Depth-first construction of a prefix tree: at depth k try every customer not yet used. No pruning, so every complete tour is produced exactly once.
Evaluation	Cost is accumulated incrementally along the path (add the leg to the new customer); at depth n the return leg to the depot is added.
Best tour	Keep (best cost, copy of best path); replace when a strictly smaller total is found.
Prediction	The first position has n choices, the next n−1, …: **n! directed tours**. (n!/2 distinct if direction is ignored, but the algorithm enumerates both directions.) Each extra customer multiplies the count by (n+1).

exhaustive() in task6.py

```
def exhaustive(dist, n):
    best = [inf, None]; count = [0]; used = [False]*(n+1); path = []
    def rec(last, cost):
        if len(path) == n:                      # complete tour
            count[0] += 1; total = cost + dist[last][0]
            if total < best[0]: best[:] = [total, path[:]]
            return
        for c in range(1, n+1):
            if not used[c]:
                used[c] = True; path.append(c)
                rec(c, cost + dist[last][c])
                path.pop(); used[c] = False
    rec(0, 0.0)
    return best[0], best[1], count[0]
```

6.2 Correctness check

(a) Analytically verifiable instance. The depot and 8 customers are placed at the vertices of a regular 9-gon of radius 50, with scrambled labels. The shortest closed tour visiting all vertices is the polygon itself, of length $9 \cdot 2R \sin(\pi/9)$ = **307.8181**. The exhaustive search (40,320 = 8! tours) returned **307.8181** (absolute difference 5.7e-14).

(b) Independent implementation. On random instances the result was compared with a separate brute-force based on `itertools.permutations`:

Table 26. Cross-check against an independent brute force.

Customers n	My exhaustive optimum	itertools optimum	Equal?	Tours examined
5	208.6311	208.6311	Yes	120
6	330.4430	330.4430	Yes	720
7	272.3679	272.3679	Yes	5,040
8	301.4531	301.4531	Yes	40,320

6.3 Growth: prediction and validation

Table 27. Candidate tours and time for seven sizes (random instances, Python).

Customers n	Predicted n!	Tours counted	Equal?	Time (s)	Tours per second	Time ratio to previous n
4	24	24	Yes	5.1e-5	470,358	–
5	120	120	Yes	1.9e-4	621,208	3.8
6	720	720	Yes	1.1e-3	640,237	5.8
7	5,040	5,040	Yes	8.1e-3	619,216	7.2
8	40,320	40,320	Yes	0.067	598,959	8.3
9	362,880	362,880	Yes	0.344	1,055,549	5.1
10	3,628,800	3,628,800	Yes	3.445	1,053,349	10.0

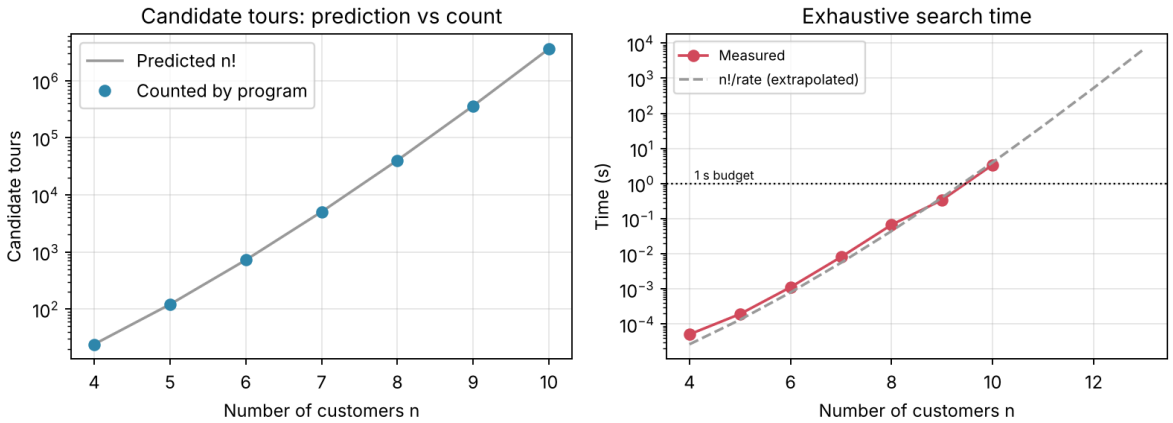


Figure 13. Left: predicted n! against counted tours. Right: measured time and extrapolation n!/rate against the 1-second budget.

The count equals n! for all seven sizes, and the time ratio between consecutive sizes approaches n + 1 (≈ 10 from n=9 to n=10: 0.34 s → 3.4 s), as predicted; the throughput stays near one million tours per second.

6.4 Local search alternative

Method: 2-opt hill climbing. A move reverses a segment of the tour (replacing two edges by two others); the best improving move is taken until no 2-opt move shortens the tour (a local minimum). Variant A: **single start** from a random tour. Variant B: **10 random restarts**, keeping the best. For n = 8, 9, 10 I generated 5 random instances each, computed the exact optimum exhaustively, and ran each variant 20 times per instance (100 runs per size).

Hill climbing with 2-opt (condensed)

```
def two_opt_climb(dist, tour):
    t = [0] + tour + [0]; improved = True
    while improved:
        improved = False; best_delta = -1e-9
        for i in 1..n-1:  for j in i+1..n:
            delta = d[t[i-1]][t[j]] + d[t[i]][t[j+1]] - d[t[i-1]][t[i]] - d[t[j]][t[j+1]]
            if delta < best_delta: best_delta, (bi, bj) = delta, (i, j)
        if best move found: reverse t[bi..bj]; improved = True
    return t[1:-1]
```

Table 28. 2-opt versus the exhaustive optimum (100 runs per size and variant).

n	Tours (exhaustive)	Single: mean excess	Single: worst	Single: reached optimum	Single: ms	Restarts: mean excess	Restarts: worst	Restarts: reached optimum	Restarts: ms
8	40,320	0.50%	4.5%	72/100	0.04	0.00%	0.0%	100/100	0.4

n	Tours (exhaustive)	Single: mean excess	Single: worst	Single: reached optimum	Single: ms	Restarts: mean excess	Restarts: worst	Restarts: reached optimum	Restarts: ms
9	362,880	0.17%	9.6%	81/100	0.05	0.00%	0.0%	100/100	0.4
10	3,628,800	1.29%	10.0%	77/100	0.06	0.00%	0.0%	100/100	0.6

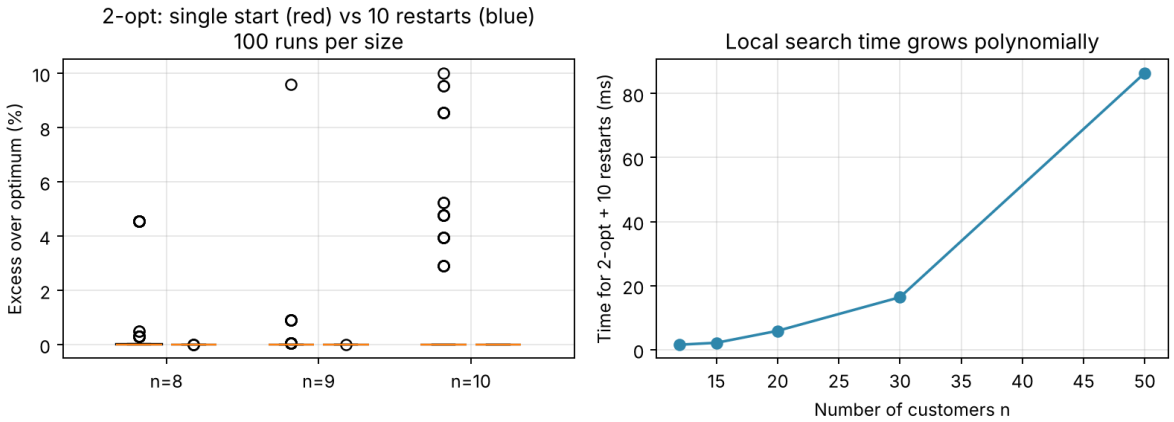


Figure 14. Left: excess over the optimum for single-start (red) and 10-restart (blue) 2-opt. Right: time of 2-opt with 10 restarts as n grows.

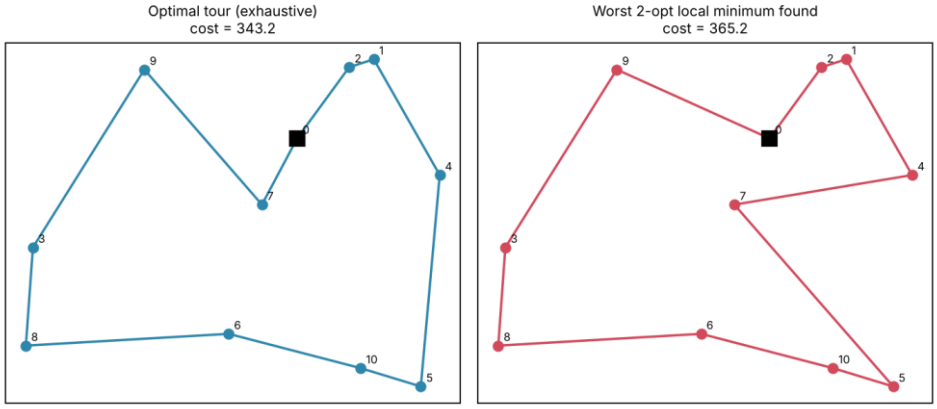


Figure 15. Optimal tour (cost 343.2) and the worst of 40 2-opt local minima for a 10-customer instance (random seed 5) where hill climbing gets stuck (cost 365.2, 6.4% longer).

Sizes beyond the exhaustive range cannot be compared with the true optimum; the reference is then the best of three long runs (60 restarts each). 2-opt with 10 restarts, 20 runs per size:

Table 29. Local search at sizes where exhaustive search is impossible.

n	Mean time (ms)	Mean excess over reference	Worst excess	Exhaustive time at 1 M tours/s
12	1.6	0.00%	0.00%	8.0 min
15	2.2	0.00%	0.00%	17 days
20	5.9	0.30%	2.96%	10^5 years
30	16.4	0.09%	1.73%	10^19 years
50	86.4	0.72%	2.43%	10^51 years

6.5 Inquiry

Why does adding a few customers cause a dramatic increase in effort?

The solution space is the set of orderings, and it grows **factorially**: every new customer can be inserted at any of the n+1 positions of every existing tour, so the number of tours is multiplied by n+1, not added to. From 10 to 12 customers the count goes from 3.6 million to 479 million (132×); 15 customers give 1.3×10^{12}, 20 give 2.4×10^{18}. Because the algorithm examines each candidate, effort is proportional to this count. Two extra customers at n = 10 therefore cost two orders of magnitude, whereas the same two customers cost a local-search method a few percent more time (1.6 ms at n = 12, 5.9 ms at n = 20).

Why can local search fail to reach the optimum?

- **Local minima:** a 2-opt neighbourhood contains only n(n−1)/2 tours reachable by reversing one segment. A tour in which no single reversal helps is a local minimum even if a better tour needs several simultaneous changes. Evidence: single-start 2-opt reached the optimum in only 72–81% of runs, and the worst local minimum in the sample picture is 6.4% above the optimum (figure above).
- **Plateaus:** when many neighbours have the same length (common with integer or grid distances) a strict-improvement rule has no uphill or downhill signal and stops. Euclidean random points rarely produce exact ties, which is why plateaus did not appear in my data.
- **Remedy shown by the data:** random restarts sample different basins; 10 restarts reached the optimum in 100/100 runs for every size up to n = 10.

6.6 Deployment decision

Response-time requirement (my choice): 1 second for a plan, so a dispatcher can re-plan interactively. Measured exhaustive throughput is 1,053,349 tours per second in Python.

Table 30. Exhaustive search against the 1-second budget (n = 9 and 10 were measured: 0.34 s and 3.4 s).

Customers n	Tours n!	Estimated time (this implementation)	If 100× faster (compiled code)
9	362,880	0.34 s	0.00 s
10	3,628,800	3.45 s	0.03 s
11	39,916,800	37.90 s	0.38 s
12	479,001,600	454.74 s	4.55 s
13	6,227,020,800	5911.64 s	59.12 s

Decision

Deploy exhaustive search for n ≤ 9 customers (0.34 s measured at n = 9, one third of the budget; n = 8 takes 0.07 s). **Switch to local search from n = 10**, where exhaustive search already takes 3.4 s (3.4× over budget) and each further customer multiplies the time by 11, 12, 13….

Local search is the only option beyond that: 2-opt with 10 restarts takes 2 ms at n = 15, 6 ms at n = 20 and 86 ms at n = 50, with mean excess ≤ 0.3% at n ≤ 30 and 0.7% at n = 50 against the best known tour.

A compiled implementation (≈ 100× faster) would only move the switching point from n = 9 to n = 11, because a factor 100 is absorbed by two extra customers. The boundary is set by the growth law, not by the hardware.

Task 7 – Diagnose, Redesign and Validate (13 marks)

Solution selected: the depth-limited minimax move chooser of Task 4 (N×N k-in-a-row game, my evaluation function E). It shows the clearest limitation: leaf evaluations are multiplied by 10–35 for every extra ply, and it already exceeds the 1-second response time at depth 5 on a 5×5 board (8.4 s). Evidence from Task 3 (move ordering decides how much alpha-beta can save) is also used.

A. Diagnose the computational bottleneck (2 marks)

Using the Task-4 counts and an instrumented run that records every position visited at each ply (4×4 to depth 5 and 5×5 to depth 4):

Table 31. Diagnosis from my own evidence.

Question	Evidence	Conclusion
Where is most effort spent?	Leaf evaluations are 95.5% of all nodes on 5×5 depth 4 (303,600 of 318,026) and 91.7% on 4×4 depth 5. Time per node is nearly constant (≈ 1 µs), so time ∝ leaves.	The effort is the number of leaf evaluations, not the cost of one evaluation (E is O(1) incrementally).
Which operation grows?	Leaves are multiplied by (c − d + 1) per extra ply: 5×5: 600 → 13,800 → 303,600 → 6,375,600.	Growth is factorial in depth: P(c,d) = c!/(c−d)!.
Is work unnecessarily repeated?	Only 83,426 distinct positions are visited among 318,026 nodes on 5×5 depth 4 (3.8× repetition); at ply 4 each position is reached 4 times, at ply 5 (4×4) 12 times.	Yes: the same position is reached by different orders of the same moves (transpositions) and evaluated from scratch each time.
What characteristic of the algorithm causes it?	It searches a **tree of move sequences**, not the **graph of positions**, and it expands every sibling in fixed cell order without any bound from what has already been found.	Two independent sources of waste: repetition (blind to transpositions) and irrelevance (no pruning).

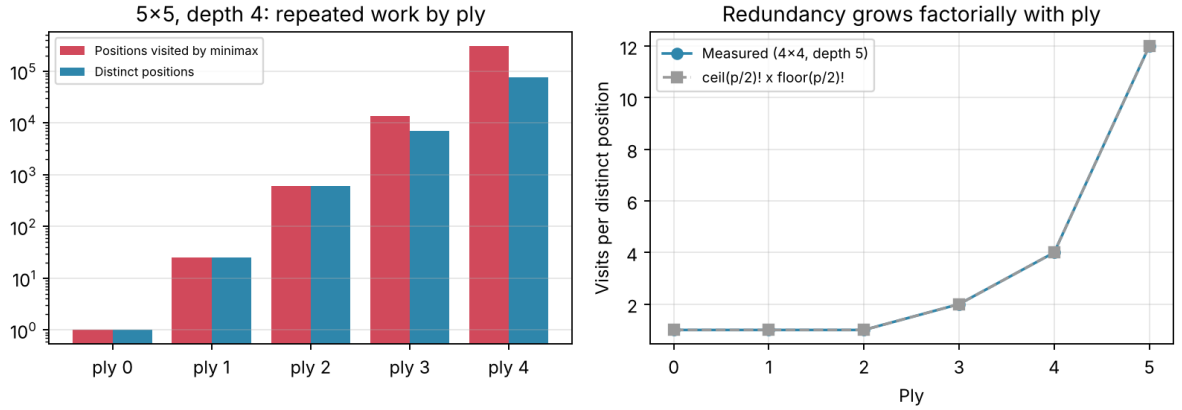


Figure 16. Left: positions visited by minimax and distinct positions per ply (5×5, depth 4). Right: visits per distinct position, measured and as predicted by ⌈p/2⌉!·⌊p/2⌋! (4×4, depth 5).

The redundancy at ply p has a closed form: X has made ⌈p/2⌉ and O ⌊p/2⌋ moves, each side's moves can be ordered in any way, so a position is reached ⌈p/2⌉!·⌊p/2⌋! times. The measured ratios (1, 1, 2, 4, 12 for plies 1–5 on 4×4) equal this formula.

B. Design objective (2 marks)

Before proposing anything, the redesign must satisfy these measurable targets, tested on the same positions as the original (the empty board plus four random positions with 4 stones placed):

Table 32. Design objectives with measurable targets.

#	Objective	Target
O1	Correctness	Same root minimax value as plain minimax in 100% of test positions.

#	Objective	Target
O2	Fewer leaf evaluations (primary metric)	≥ 10× fewer at 5×5 depth 4 and ≥ 30× fewer at 5×5 depth 5.
O3	Total work, honest accounting	Leaf evaluations **plus** any extra evaluations spent on ordering must fall by ≥ 5× at 5×5 depth 5.
O4	Response time	Reach at least depth 6 on the 5×5 board within the 1 s limit (the original manages depth 4).
O5	Bounded extra memory	Table size small compared with the original's node count (≤ 5% of nodes visited).

C. The revised computational strategy (4 marks)

The diagnosis names three separate problems, and I address each with a property of the game rather than by importing a named method:

1. **Reuse (property: the value of a position does not depend on how it was reached).** Give each position a 64-bit signature by XOR-ing random numbers assigned to (cell, player) pairs; XOR is order-independent, so all move orders of the same stones give the same key and the signature is updated in O(1) per move. Store the value of every internal position in a table. Because the search is cut off by alpha-beta, a stored value may only be a **bound**; it is therefore saved with a flag (exact / lower bound / upper bound) relative to the window in which it was computed, and used only as far as that flag allows.
2. **Bound (property: only the maximum at MAX nodes and the minimum at MIN nodes matter).** Pass a window (α, β) down the tree. A branch is abandoned as soon as it cannot influence the decision: when the opponent already has a better alternative earlier on the path (α ≥ β).
3. **Order (property from Task 3: the saving of bounds depends on how early the best move is tried).** At each internal node, look one ply ahead using E itself: make each move, read the incremental score, undo, and try moves in descending order of this score for the player to move (immediate wins first). These **ordering probes** are an extra cost, counted separately and included in the total work.

Design (pseudocode)

```
SEARCH(depth, player, alpha, beta, key):
    if depth == 0: return E(position)                       # leaf (not stored: E is O(1))
    entry <- table[key]
    if entry: if entry.exact: return entry.value
              if entry.lower: alpha <- max(alpha, entry.value)
              if entry.upper: beta  <- min(beta,  entry.value)
              if alpha >= beta: return entry.value
    a0, b0 <- alpha, beta
    moves <- empty cells sorted by one-ply score for `player`  # ordering probes
    for m in moves:
        v <- terminal value if m wins
             else SEARCH(depth-1, -player, alpha, beta, key xor Z[m][player])
        update best, alpha (MAX) or beta (MIN)
        if alpha >= beta: break                              # cut-off
    table[key] <- (best, UPPER if best <= a0 else LOWER if best >= b0 else EXACT)
    return best
```

Table 33. Summary of the design.

Aspect	Statement
What changed	Tree search → search of positions with bounds and ordered children.
Property exploited	Value depends only on the position (transpositions); the decision depends only on the best move (bounds); cut-offs are efficient only with best-first order.
Operations avoided	Re-searching positions already solved; evaluating subtrees refuted by an earlier move.
Preprocessing / extra memory	One random 64-bit table (2 × cells numbers). A hash table of positions visited: 2,075 to 16,941 entries in my runs (below).

Aspect	Statement
New trade-offs	(1) Ordering probes cost ≈ b extra make/undo operations per internal node. (2) Table memory and hashing time. (3) Complexity of bound flags (a source of subtle bugs). (4) Only the best root move is exact; other root moves return bounds, so random tie-breaking among equal moves is lost. (5) A 64-bit signature can in theory collide (probability ≈ $N^2/2^{65} \approx 10^{-8}$ for 10^5 positions).

D. Predictions before implementation (1 mark)

- **P1** The redesign returns exactly the same root value as minimax in every test position.
- **P2** Reuse alone should reduce leaf evaluations by about the repetition factor $\lceil d/2 \rceil! \cdot \lfloor d/2 \rfloor!$: ≈ 4× at depth 4 and ≈ 12× at depth 5.
- **P3** Bounds alone with the natural cell order should help little (the order is arbitrary).
- **P4** Ordering is what makes bounds effective: leaf evaluations should fall by more than 10× at depth 4 and by more than 30× at depth 5 relative to minimax; total work (including probes) falls less.
- **P5** The improvement is in **leaf evaluations and time**; playing strength should not change at equal depth.

E. Implementation and validation (2 marks)

Five variants were run on **identical positions** (empty board plus four random 4-stone positions): V0 plain minimax; V1 minimax with a position table (exact values only); V2 alpha-beta, natural cell order; V3 alpha-beta with ordering; V4 the full design (ordering + table + bounds).

Table 34. Ablation on 5×5 (k=4): mean over five positions. All variants returned the same value in every position.

Test	Variant	Leaf evals	Ordering probes	Total work	Table hits	Time (ms)	Leaf-eval reduction	Total-work reduction
5x5 depth 4	V0 minimax	175,358	0	175,358	0	246.4	–	–
	V1 minimax+memo	87,679	0	87,679	4,568	173.5	2.0×	2.0×
	V2 alpha-beta	18,099	0	18,099	0	28.9	9.7×	9.7×
	V3 alpha-beta+ordering	1,341	12,662	14,003	0	19.7	130.8×	12.5×
	V4 ordering+memo+alpha-beta	**1,204**	**9,727**	**10,931**	**151**	**15.9**	**145.6×**	**16.0×**
5x5 depth 5	V0 minimax	3,210,523	0	3,210,523	0	4496.4	–	–
	V1 minimax+memo	802,631	0	802,631	48,408	1415.0	4.0×	4.0×
	V2 alpha-beta	162,230	0	162,230	0	266.2	19.8×	19.8×
	V3 alpha-beta+ordering	9,284	40,305	49,588	0	75.6	345.8×	64.7×
	V4 ordering+memo+alpha-beta	**6,325**	**27,909**	**34,235**	**419**	**49.7**	**507.6×**	**93.8×**

Table 35. Original versus redesign on all test conditions (means of five positions; all 160 value comparisons with plain minimax were equal).

Board	Depth	V0 leaf evals	V4 leaf evals	V4 probes	Leaf reduction	Total-work reduction	V0 ms	V4 ms	Same value?
4x4 (k=4)	4	18,060	326	2,264	55.5×	7.0×	18.7	3.3	Yes
4x4 (k=4)	5	177,420	996	4,229	178.1×	34.0×	202.2	7.6	Yes
4x4 (k=4)	6	1,643,537	2,236	13,504	735.0×	104.4×	1909.3	19.9	Yes
5x5 (k=4)	3	9,136	559	1,877	16.3×	3.8×	11.7	3.3	Yes
5x5 (k=4)	4	175,358	1,204	9,727	145.6×	16.0×	246.4	15.9	Yes
5x5 (k=4)	5	3,210,523	6,325	27,909	507.6×	93.8×	4496.4	49.7	Yes

Board	Depth	V0 leaf evals	V4 leaf evals	V4 probes	Leaf reduction	Total-work reduction	V0 ms	V4 ms	Same value?
6x6 (k=4)	3	32,340	1,136	3,924	28.5×	6.4×	46.1	7.8	Yes
6x6 (k=4)	4	972,132	2,640	29,841	368.2×	29.9×	1565.5	59.5	Yes

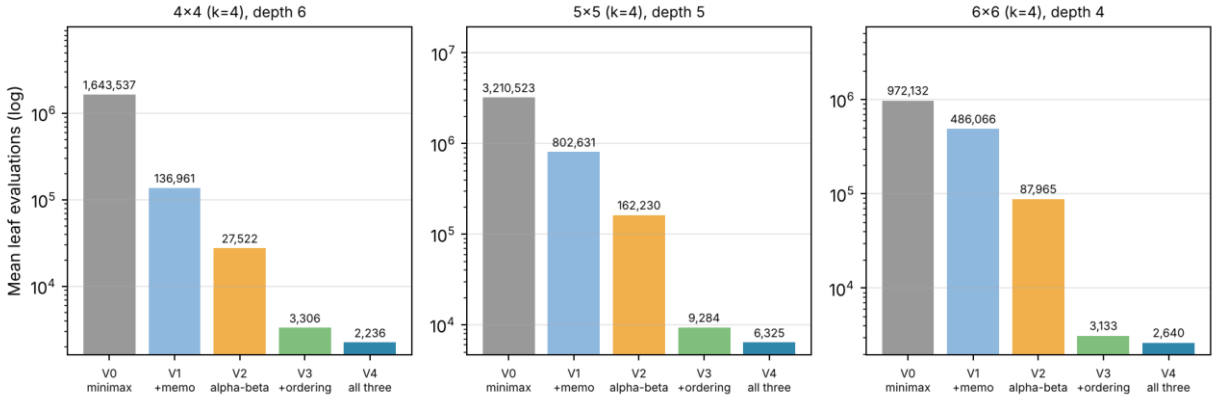


Figure 17. Mean leaf evaluations (log scale) of the five variants in three test conditions.

Table 36. Depth reached by the redesign from the empty board (stopped at 12 million nodes).

Board (empty)	Depth	Nodes	Leaf evals	Ordering probes	Time (s)	Value
4x4 (k=4)	6	8,407	5,229	29,208	0.041	-1
4x4 (k=4)	7	41,468	27,409	111,453	0.223	11
4x4 (k=4)	8	125,410	64,540	396,712	0.864	-2
4x4 (k=4)	9	187,253	98,199	511,851	0.824	11
5x5 (k=4)	5	13,216	10,493	44,403	0.088	31
5x5 (k=4)	6	20,233	11,905	112,047	0.200	0
5x5 (k=4)	7	118,806	89,753	330,496	0.769	91
6x6 (k=4)	4	3,516	2,045	28,989	0.051	-7
6x6 (k=4)	5	31,980	27,911	94,446	0.197	89
6x6 (k=4)	6	86,464	53,500	627,484	1.194	-15

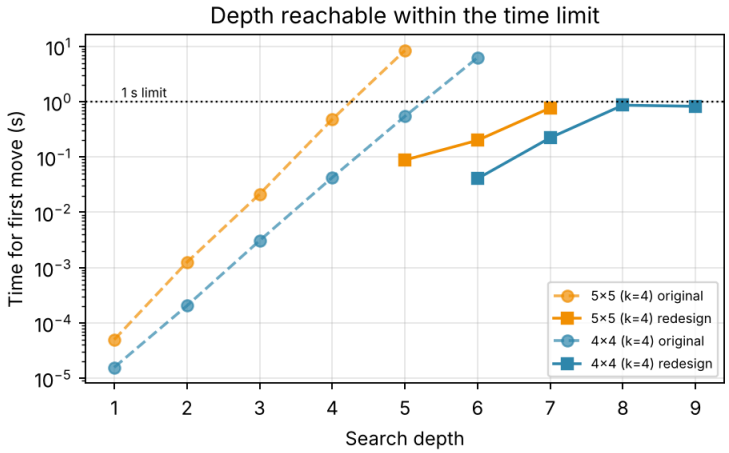


Figure 18. Time for the first move against depth: original (dashed) and redesign (solid).

Table size (O5): 4x4 (k=4) depth 6: 2,559 entries (30.4% of 8,407 nodes); 5x5 (k=4) depth 5: 2,075 entries (15.7% of 13,216 nodes); 5x5 (k=4) depth 7: 16,941 entries (14.3% of 118,806 nodes); 6x6 (k=4) depth 5: 2,907 entries (9.1% of 31,980 nodes).

Strength check

Table 37. 5×5 (k=4), 20 games per row, colours alternating. V0 = plain minimax; V4 = redesign. The first row (identical agents) shows the sampling noise of 20 games.

Agent A	Agent B	A wins	Draws	A losses	A ms/move	B ms/move
V0 depth 3	V0 depth 3	13	0	7	9.6	10.0
V4 depth 4	V0 depth 3	7	13	0	27.9	5.9
V4 depth 5	V0 depth 3	13	6	1	141.9	7.2
V4 depth 5	V0 depth 4	0	14	6	121.8	100.7
V4 depth 4	V0 depth 4	0	20	0	22.9	81.7
V4 depth 6	V0 depth 4	0	20	0	277.5	85.5
V4 depth 6	V0 depth 2	2	18	0	290.1	0.3
V0 depth 4	V0 depth 2	6	14	0	99.9	0.3

F. Defending the final design (2 marks)

Table 38. Objectives versus outcome.

Objective	Target	Result	Met?
O1 Correctness	Same value in all positions	Identical in every comparison (8 configurations × 5 positions × 4 alternative variants = 160 comparisons)	Yes
O2 Leaf evaluations	≥ 10× (d=4), ≥ 30× (d=5)	146× (d=4: 175,358 → 1,204); 508× (d=5: 3,210,523 → 6,325)	Yes, far exceeded
O3 Total work incl. probes	≥ 5× at d=5	94× (3,210,523 → 34,235); at d=4 16×	Yes
O4 Depth within 1 s	≥ 6 on 5×5	Depth 7 in 0.77 s (original: depth 4)	Yes
O5 Memory	≤ 5% of nodes	2,075 entries for 13,216 nodes (15.7%) at 5×5 depth 5; 14% at depth 7	**No** (but absolute size is tiny: ≈ 17,000 entries)

Were the predictions supported?

- **P1 supported:** values identical everywhere.
- **P2 partly wrong.** The table alone gave 2.0× at depth 4 and 4.0× at depth 5 (175,358 → 87,679; 3,210,523 → 802,631), not 4× and 12×. The reason is a one-ply error in my prediction: leaves are not stored, so only repetition among the positions at ply d−1 is removed ($\lceil(d-1)/2\rceil!\cdot\lfloor(d-1)/2\rfloor!$ = 2 and 4), exactly the observed factors. I left leaves out on purpose because E is O(1); at depth 3 the table therefore has no effect (9,136 evaluations in both).
- **P3 wrong in magnitude:** bounds alone with natural order already cut leaf evaluations 9.7× at depth 4 and 19.8× at depth 5. The cell order is arbitrary but not adversarial, so half-decent cut-offs occur by chance; Task 3 showed the same for random order.
- **P4 supported:** ordering raises the saving from 20× to 346× at depth 5 (162,230 → 9,284 leaves), and adding the table gives 508× (6,325 leaves). The total-work reduction is smaller (94×) because ordering probes cost 4.4× as many operations as the leaves they save (27,909 vs 6,325).
- **P5 supported:** at equal depth 4 the redesign drew all 20 games against minimax, while taking 23 ms per move instead of 82 ms.

Magnitude of improvement and new costs

At 5×5 depth 5 the first move falls from 4.5 s to 0.050 s (90×); on 4×4 the redesign searches **depth 9 in 0.82 s** where the original needed 6.3 s for depth 6; on 5×5 it reaches depth 7 within the limit. The price is the memory of a position table (≤ 17,000 entries here), the ordering probes (4–11× the number of leaf evaluations), more

complex code and the loss of exact values for non-best root moves. The memory objective O5 missed its target ratio because with a cut-off search the number of nodes falls faster than the number of distinct positions, but the table stays tiny in absolute terms.

Does extra depth buy strength?

Not with this evaluation function. Depth 6 drew all 20 games against depth 4 and beat depth 2 only 2 times in 20; depth 5 lost 6 of 20 games to depth 4, although it beat depth 3 in 13 (1 loss). Odd depths end the search just after the searching player's own move, when E tends to be optimistic (a horizon effect), so depth 4 played better than depth 5. The redesign has solved the **speed** limitation; the next limit is the **quality of E** (a better evaluation, or extending threats at the horizon), which is outside this redesign.

When the original may still be preferable

- **Very shallow search** (depth ≤ 2–3) or tiny boards: the original is simpler and the gain is a few milliseconds (depth 3 on 5×5: 12 ms vs 3 ms).
- **When all root moves need exact values** (move rankings, random tie-breaking, training targets): alpha-beta returns only bounds for the non-best moves.
- **When E is very expensive or ordering information is poor:** probes then cost more than they save; and in memory-constrained or safety-critical code the plain algorithm is easier to verify.

Summary of Conclusions and Reproducibility

Table 39. Findings and the predictions that failed.

Task	Principal conclusion	Prediction that was wrong (and why)
1	A* with Manhattan distance for the 95% open-road profile: 22× fewer expansions than UCS, always optimal; greedy is optimal on only 44.5% of trips.	A* would expand more than greedy in the trap map – both expanded the identical 460 cells.
2	Online DFS is acceptable below ≈ 200–500 cells and for one-off missions; with static maps, explore once and reuse the map (80% fewer moves over 40 missions).	Loops reduce revisits but make the competitive ratio worse, not better.
3	Alpha-beta costs 127 to 4,096 leaves on the same b=4, d=6 shape; b and d alone do not determine its cost.	Reverse-sorting is not a true worst case (87% of leaves, not 100%); random order was 1.7× the rule-of-thumb estimate.
4	Node counts equal the derived c!/(c−d)! sums exactly; plain minimax reaches depth 4–5 within 1 s.	Win rate against a random player does not rise with depth – the metric saturates.
5	Arc-consistency propagation makes backtracking nearly backtrack-free (≈ n nodes) but is incomplete and can cost more checks on small problems.	None material; the adversarial K4 instance confirmed the predicted incompleteness.
6	n! growth confirmed; deploy exhaustive search up to 9 customers, 2-opt with restarts beyond.	None material.
7	Position reuse + bounds + ordering cut leaf evaluations 146× (depth 4) and 508× (depth 5) with identical values.	Table alone gave 2×/4×, not 4×/12×; bounds alone gave 20×, not “little”; memory target missed.

Files submitted with this report

File	Content
code/task1.py … task7.py, task4b.py, task7b.py, task7c.py, game.py	Source code of every experiment (game.py holds the shared board engine and evaluation function).
code/task*_fig.py	Scripts that draw the figures from the stored data.
data/*.json	Raw experimental data behind every table and figure.
figures/*.png	All figures used in the report.

To reproduce: run `python3 task1.py` … `python3 task7.py` in the code folder (about 15 minutes in total; random seeds are fixed in each script). Timings depend on the machine; counts do not.